

Rescue Operations System (ARDMS) - Technology Stack & Specifications

This document outlines the core technologies, application specifications, and hardware requirements for the Rescue Operations System (ARDMS).

1. Core Technology Stack

- **Backend:** Node.js with Express.js
- **Database:** SQLite3 (running in WAL mode for high concurrency)
- **Real-Time Communication:** Native `ws` WebSocket library
- **Frontend Architecture:** Single Page Applications (SPAs) built with HTML, CSS, and Vanilla JavaScript.
- **Mobile App Framework:** React Native (specifically leveraging `react-native-webview` to wrap the frontend SPAs).

2. Application Ecosystem & APKs

The system is deployed across three distinct applications. All mobile apps are packaged as standard Android APKs wrapping the core HTML/JS interfaces.

A. Public SOS App (`preview-mobile-app.html` -> `Public_SOS_App.apk`)

- **Purpose:** For citizens to raise SOS alerts, send live geolocation, and upload audio/image evidence.
- **Media Optimization:** Features a client-side compression engine ensuring final image payloads are max 200KB and audio recordings are max 100KB, drastically improving reliability over poor 3G/4G networks.
- **GPS Architecture:** Forces a high-accuracy, un-cached `satellite navigator.geolocation.getCurrentPosition` read upon SOS trigger.

B. Rescuer Field App (`preview-rescuer.html` -> `Rescuer_Rescue_App.apk`)

- **Purpose:** For field operatives to receive AI-assigned tasks, accept/decline missions, and stream live GPS back to headquarters.
- **Navigation Engine:** Integrates Mapbox GL JS for dynamic routing and offline map tile caching.

C. Admin & Command Center App (`raw_admin.html` -> `Admin_Rescue_App.apk` / `Web ADMIN.html`)

- **Purpose:** A comprehensive dashboard for dispatchers to view live tracking, AI assignments, and real-time statistics. Operates both as a Web App (in browsers) and an Android APK (for tablet commanders).
- **Auto-Refresh:** All configuration changes (like interval tweaks) trigger automatic WebSocket UI refreshes instantly across all connected Admin panels.

3. Hardware & OS Requirements for Mobile APKs

Rescuer App

- **OS Requirement:** Android 7.0 (Nougat) or higher
- **RAM:** Minimum 2 GB
- **CPU:** Quad-Core Processor (1.4 GHz or higher)
- **Permissions Required:** Foreground & Background Location (GPS), Camera, Microphone, Network Access.

Public App

- **OS Requirement:** Android 7.0 (Nougat) or higher
- **RAM:** Minimum 2 GB
- **CPU:** Quad-Core Processor (1.4 GHz or higher)
- **Permissions Required:** Location (GPS), Camera, Microphone, Network Access.

Admin App (Tablet/Mobile)

- **OS Requirement:** Android 8.0 (Oreo) or higher
- **RAM:** Minimum 3 GB (4 GB recommended for rendering heavy map layers and multiple markers)
- **CPU:** Octa-Core Processor recommended.
- **Permissions Required:** Network Access, Storage (for downloading logs).

Rescue Operations System (ARDMS) - Deployment Guide

This document exclusively covers the setup, network configuration, and compilation protocols for the ARDMS emergency management network.

1. Prerequisites

Ensure the host deployment machine (Local Server) has the following installed:

- **Node.js:** Version 18+ (for the backend server)
- **Python:** Version 3.8+ (for synchronization and build automation scripts)
- **Java Development Kit (JDK):** Version 17 (Required by React Native/Gradle)
- **Android SDK:** Available via Android Studio (with environment variables `ANDROID_HOME` correctly set)

2. Setting Up the Local Backend

1. Initialize Dependencies:

Navigate to the `system-backend` directory and install NPM packages:

```
cd system-backend
npm install
```

2. Launch the Server:

```
node server.js
```

The backend will auto-detect the local Wi-Fi/LAN IP address (e.g., `192.168.x.x`) and bind to it. It will also serve the static Web Admin dashboard at `http://<IP_ADDRESS>:3000`.

3. Synchronizing Frontend Edits

The React Native mobile apps directly wrap HTML files located in the root directory. If you modify `preview-mobile-app.html`, `preview-rescuer.html`, or `raw_admin.html`, you **MUST** synchronize these files into the React Native build pipeline before compiling.

Run the synchronization script from the root project directory:

```
python sync_apps.py
```

This script will parse the HTML, inject the detected Local IP Address into the JavaScript payloads, and copy them into the respective React Native project folders as `htmlStr.js`.

4. Compiling the APKs

After synchronization, you can compile the APKs using the provided automated Python scripts. These scripts orchestrate Gradle and React Native CLI commands.

- **Build Public App:**

```
python build_public_apk.py
```

• **Build Rescuer App:**

```
python build_rescuer.py
```

• **Build Admin App:**

```
python build_admin_apk.py
```

(Note: Compiling the Admin APK is only necessary if you require tablet deployments for commanders. Otherwise, they can just use the Web Admin via a browser).

Upon successful completion, the compiled, release-ready APKs will be automatically moved to the `Output_APKs/` directory.

5. Network Reconnection & App Configurations

All compiled APKs feature an offline-capable manual IP entry system.

- If the central backend server changes its Wi-Fi network (and therefore its IP address), **you do not need to recompile the APKs.**
- Operators and citizens can simply tap the server connection status icon in the app, type the new backend IP address, and seamlessly reconnect to the websocket pipeline.

Rescue Operations System (ARDMS) - Project Architecture & Workflow

This document explains the overarching system architecture, logic loops, and the agentic workflow (for AI assistants modifying this codebase).

1. System Architecture & Core Files

- **system-backend/server.js:** The central nervous system. Handles REST API requests, SQLite database operations, and acts as the master WebSocket broadcaster.
- **preview-mobile-app.html:** The Public SOS interface. Collects citizen data, compresses media locally, fetches high-accuracy GPS coordinates, and POSTs to `/api/rescue-request`.
- **preview-rescuer.html:** The Field Responder interface. Maintains a persistent WebSocket connection, listens for `NEW_COMMAND`, and POSTs live location data every 10 seconds.
- **raw_admin.html (Compiled to Web_ADMIN.html):** The Headquarters interface. Consumes the WebSocket feed to instantly update maps, tables, and AI configuration settings in real-time.

2. Artificial Intelligence Engine Workflow

The system incorporates a dynamic AI assignment loop inside `server.js` designed to automate emergency dispatching:

1. **Trigger Phase:** The AI Timer (configurable via the Web Admin) polls the `rescue_requests` table for any `status = 'pending'`.
2. **Proximity Phase:** For each pending SOS, the engine queries the `users` table for online rescuers. It calculates the Haversine distance between the rescuer and the SOS location.
3. **Filtering Phase:** It filters out any rescuers currently occupied (e.g., `status = 'busy'`), and ignores rescuers who have already been assigned to this specific SOS (recorded in `sos_assignment_history`).
4. **Assignment Phase:** It dispatches the task to the nearest eligible rescuer via a targeted WebSocket `NEW_COMMAND` event.
5. **Reassignment Loop:** If a rescuer ignores the command for a set duration (Buffer Time), the AI marks them as ignored in the history, reverts the task to `pending`, and the loop restarts, guaranteeing the SOS is dynamically passed down the chain of command until accepted.

3. Data Integrity & Concurrency

- **Database Locks:** `server.js` implements active memory locks (`activeSubmissions`, `activeCommandUpdates`) to intercept duplicate network requests from users frantically mashing submission buttons.
- **Auto-Refresh Protocols:** Configuration changes broadcast `SETTINGS_UPDATED`, immediately triggering silent, under-the-hood API fetches on all connected Admin clients to keep UIs perfectly synchronized without manual page refreshes.

4. Agentic Workflow (For Antigravity & AI Developers)

If you are an AI assistant or new developer modifying this project:

1. **Always read this file first** to understand the system layout.
2. Edit the HTML files (`preview-*.html` or `raw_admin.html`) and the backend logic (`server.js`) directly.
3. **NEVER manually edit `htmlStr.js`.** Always use the Python build scripts (e.g., `python sync_apps.py`) to inject your HTML changes into the React Native shells.
4. Adhere to the established fallback UI logic: Ensure graceful failures for GPS timeouts, WebSocket disconnects, or database lock rejections (e.g., Error 429).